PYTHON EXCEPTION HANDLING

What is an Exception?

- An **exception** is an abnormal condition that occurs **during the execution of a program**.
- It interrupts the **normal flow of instructions** in a program.
- Exceptions occur **only at runtime**, not during compilation.
- If exceptions are not handled properly, the program **terminates suddenly**.
- Python treats exceptions as **objects**, which contain error information.

Example

- ATM card blocked → exception
- Internet disconnected → exception
- File not found → exception

Why Exception Handling is Required?

- Exception handling is required to make programs **robust and fault-tolerant**.
- Without exception handling, a single error can **crash the entire application**.
- Handling exceptions ensures that programs **continue execution safely**.
- It allows programmers to display **meaningful error messages** instead of technical errors.
- Exception handling is essential in **real-world applications** like banking, file handling, and web apps.

Without Exception Handling

Program stops immediately when error occurs.

With Exception Handling

Program handles error gracefully and continues execution.

Types of Errors in Python :

1) Syntax Errors

- Syntax errors occur when Python grammar rules are violated.
- They are detected **before program execution**.
- The program will not run until syntax errors are fixed.

Eample code:

```python
if x > 5
    print(x)
```

---

2) Runtime Errors (Exceptions)

- Runtime errors occur **while the program is running**.
- They happen due to invalid operations such as dividing by zero.
- These errors can be handled using exception handling.

Eample code:

```python
print(10 / 0)
```

---

3) Built-in Exceptions in Python

- Python provides many built-in exception classes.
- Each exception represents a **specific type of error**.
- Using built-in exceptions helps in **accurate error handling**.
- These exceptions are automatically raised by Python.

❖ ZeroDivisionError:

- Occurs when a number is divided by **zero**.

Eample code:

```python
a = 10
b = 0
print(a / b)
```

Output:

```vbnet
ZeroDivisionError: division by zero
```

- Division by zero is mathematically invalid, so Python raises this exception.

❖ ValueError:

- Occurs when a function receives a **correct data type but invalid value**.

Eample code:

```python
x = int("abc")
print(x)
```

Output:

```csharp
ValueError: invalid literal for int()
```

- "abc" cannot be converted into an integer, so Python raises ValueError.

❖ TypeError:

- Occurs when an operation is performed on **incompatible data types**.

Eample code:

```python
a = 10
b = "5"
print(a + b)
```

Output:

```bash
TypeError: unsupported operand type(s)
```

- Python cannot add an integer and a string directly.

IndexError:

Occurs when accessing an **index that does not exist** in a list or tuple.

Eample code:

```python
nums = [10, 20, 30]
print(nums[5])
```

Output:

```pgsql
IndexError: list index out of range
```

- ➢ The list has only 3 elements, but index 5 does not exist.

❖ KeyError:

- Occurs when trying to access a **missing key** in a dictionary.

Eample code:

```python
student = {"name": "Ravi", "age": 20}
print(student["marks"])
```

Output:

```vbnet
KeyError: 'marks'
```

- ➢ The key "marks" is not present in the dictionary.

❖ FileNotFoundError:

- Occurs when a file **does not exist** in the given path.

Eample code:

```python
f = open("data.txt", "r")
```

Output:

```yaml
FileNotFoundError: No such file or directory
```

- Python cannot find the file data.txt, so it raises this exception.

❖ NameError:

- Occurs when a **variable is used before defining it**.

Eample code:

```python
print(x)
```

Output:

```pgsql
NameError: name 'x' is not defined
```

- Variable x was never created before use.

❖ ImportError

- Occurs when Python **fails to import a module**.

Eample code:

```python
import abcdef
```

Output:

```vbnet
ImportError: No module named 'abcdef'
```

- The module abcdef does not exist in Python.

---

4) Basic Exception Handling (try-except)

- The try block contains code that **may cause an exception**.
- The except block contains code to **handle the exception**.
- If an exception occurs in try, control is transferred to except.
- If no exception occurs, except is skipped.
- This prevents the program from crashing.

Eample code:

```python
try:
    num = int(input("Enter a number: "))
    print(10 / num)
except:
    print("An error occurred")
```

---

5) Handling Specific Exceptions:

- Handling specific exceptions is considered a **best programming practice**.
- It helps identify the exact cause of the error.
- Specific handling avoids catching unwanted exceptions.
- It improves code clarity and debugging efficiency.

Eample code:

```python
try:
    print(10 / 0)
except ZeroDivisionError:
    print("Division by zero is not allowed")
```

---

6) Multiple except Blocks:

- Python allows multiple except blocks for a single try.
- Each except block handles a **different type of exception**.
- Only the matching exception block is executed.
- This approach provides **fine-grained control** over errors.

Eample code:

```python
try:
    x = int("abc")
except ValueError:
    print("Value conversion error")
except TypeError:
    print("Type mismatch error")
```

---

7) else Block:

- The else block executes **only when no exception occurs**.
- It contains code that should run when try is successful.
- Using else improves program readability.
- It separates normal logic from error-handling logic.

Eample code:

```python
try:
    print(10 / 2)
except ZeroDivisionError:
    print("Error occurred")
else:
    print("Operation successful")
```

---

8) finally Block

- The finally block always executes, **regardless of exceptions**.
- It is used for **cleanup operations**.
- Resources like files and database connections are released here.
- It ensures that critical code is executed.

Eample code:

```python
try:
    print(10 / 0)
except:
    print("Error")
finally:
    print("Execution completed")
```

---

9) Nested try-except:

- Nested exception handling means **placing one try-except inside another**.
- It is used for handling **multiple risky operations separately**.
- It allows more precise error control.
- However, excessive nesting should be avoided.

Eample code:

```python
try:
    try:
        print(10 / 0)
    except ZeroDivisionError:
        print("Inner exception handled")
except:
    print("Outer exception handled")
```

---

10) Raising Exceptions (raise):

- The raise keyword is used to **force an exception**.
- It is helpful for validating user input.
- It allows programmers to stop program execution intentionally.
- raise is commonly used in real-world applications.

Eample code:

```python
age = 16
if age < 18:
    raise ValueError("Age must be 18 or above")
```

---

11) User-Defined Exceptions :

- User-defined exceptions are **custom exceptions** created by programmers.
- They inherit from the built-in Exception class.
- They provide meaningful error messages.
- They are used when built-in exceptions are not sufficient.

Eample code:

```python
class InvalidAgeError(Exception):
    pass

try:
    age = int(input("Enter age: "))
    if age < 18:
        raise InvalidAgeError("Not eligible to vote")
except InvalidAgeError as e:
    print(e)
```

---

12) Exception Handling with Files

- File operations are **error-prone**.
- Files may not exist or may have permission issues.
- Exception handling ensures safe file operations.
- It prevents program crashes during file access.

Eample code:

```python
try:
    f = open("data.txt", "r")
    print(f.read())
except FileNotFoundError:
    print("File not found")
finally:
    f.close()
```

---

13)With Statement :

- The with statement automatically manages resources.
- It ensures that files are properly closed.

- It reduces code length and complexity.
- It is safer than manual file handling.

Eample code:

```python
try:
    with open("data.txt", "r") as f:
        print(f.read())
except FileNotFoundError:
    print("File missing")
```

Mini Project Using Exception Handling

Project Title:

User Input Validation System Using Exception Handling

Project Description:

This project demonstrates the use of Python exception handling to manage runtime errors. It validates user input and handles common exceptions such as invalid input, division by zero, and unexpected errors. The project uses try, except, else, and finally blocks to prevent program crashes and display user-friendly error messages. This helps in understanding how exception handling improves program reliability and safety.

Eample code:

python

```python
class ATM:
    def __init__(self, balance=5000):
        self.balance = balance

    def check_balance(self):
        print("Your current balance is:", self.balance)

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self.balance += amount
        print("Amount deposited successfully")

    def withdraw(self, amount):
        if amount <= 0:
            raise ValueError("Withdraw amount must be positive")
        if amount > self.balance:
            raise ValueError("Insufficient balance")
        self.balance -= amount
        print("Please collect your cash")
```

```python
atm = ATM()

while True:
    print("\n----- ATM MENU -----")
    print("1. Check Balance")
    print("2. Deposit")
    print("3. Withdraw")
    print("4. Exit")

    try:
        choice = int(input("Enter your choice: "))

        if choice == 1:
            atm.check_balance()

        elif choice == 2:
            amount = float(input("Enter amount to deposit: "))
            atm.deposit(amount)

        elif choice == 3:
            amount = float(input("Enter amount to withdraw: "))
            atm.withdraw(amount)

        elif choice == 4:
            print("Thank you for using ATM")
            break

        else:
```

```python
        else:
            print("Invalid choice. Please select 1-4")

    except ValueError as e:
        print("Error:", e)

    except Exception:
        print("Unexpected error occurred")

    finally:
        print("Transaction completed")
```